Không hề khó, ngược lại còn **dễ thở hơn rất nhiều** so với cách viết thuần (vanilla). Bản chất của `CommunityToolkit.Mvvm` là dùng **Source Generators** để tự động sinh ra toàn bộ đoạn code lặp đi lặp lại (boilerplate) của `INotifyPropertyChanged` và `ICommand` lúc biên dịch.

Cậu chỉ cần nhớ đúng **1 class kế thừa (** `ObservableObject` **)** và **2 Attribute chính (** `[ObservableProperty]` , `[RelayCommand]` **)** là đã giải quyết được 90% nhu cầu hằng ngày.

1. So sánh Data Binding (`INotifyPropertyChanged`)

Cách làm gốc (Viết tay `INotifyPropertyChanged`)

Mỗi thuộc tính muốn binding ra UI đều phải khai báo 1 biến `private` , 1 property `public` , và tự gọi hàm thông báo thay đổi.

C#

```
public class UserViewModel : INotifyPropertyChanged
{
    private string _userName;
    public string UserName
    {
        get => _userName;
        set
        {
            if (_userName != value)
```

```
            {
                _userName = value;
                OnPropertyChanged();
            }
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string? propertyName = nu
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName))
    }
}
```

Cách dùng `CommunityToolkit.Mvvm`

Chỉ cần gắn `[ObservableProperty]` lên biến `private` . Toolkit sẽ tự động sinh ra Property viết hoa chữ cái đầu (`UserName`) kèm đầy đủ logic `OnPropertyChanged` .

C#

```
// Lưu ý bắt buộc có từ khóa 'partial'
public partial class UserViewModel : ObservableObject
{
    [ObservableProperty]
    private string _userName;
    // Tự động sinh ra property public: UserName
}
```

(Phần code XAML `<TextBox Text="{Binding UserName}"/>` ở View giữ nguyên 100%, không đổi gì cả).

2. So sánh Command Binding (`ICommand`)

Cách làm gốc (Tự tạo class `RelayCommand` kế thừa `ICommand`)

Cậu phải tự viết một class `RelayCommand` triển khai `CanExecute` , `Execute` , `CanExecuteChanged` , sau đó khởi tạo từng command trong Constructor của ViewModel.

C#

```
public class UserViewModel : INotifyPropertyChanged
{
    public ICommand SaveCommand { get; }

    public UserViewModel()
    {
        SaveCommand = new RelayCommand(ExecuteSave, CanSave);
    }

    private bool CanSave() => !string.IsNullOrEmpty(UserName);

    private void ExecuteSave()
    {
        // Logic lưu dữ liệu
    }
    // ... (lược bỏ phần code INotifyPropertyChanged bên trên)
}
```

Cách dùng `CommunityToolkit.Mvvm`

Chỉ cần viết một hàm bình thường và gắn `[RelayCommand]`. Toolkit sẽ tự sinh ra property có hậu tố `Command` (ví dụ hàm `Save` -> sinh ra `SaveCommand`, hàm `SaveAsync` -> sinh ra `SaveCommand` hỗ trợ bất đồng bộ).

C#

```
public partial class UserViewModel : ObservableObject
{
    [ObservableProperty]
    [NotifyCanExecuteChangedFor(nameof(SaveCommand))] // Tự cập nhật trạng thái n
    private string _userName;

    private bool CanSave() => !string.IsNullOrEmpty(UserName);

    [RelayCommand(CanExecute = nameof(CanSave))]
    private async Task SaveAsync()
    {
        // Tự sinh ra SaveCommand, hỗ trợ async/await chuẩn chỉnh
```

```
        await Task.Delay(1000);
    }
}
```

3 quy tắc "nhập môn" duy nhất cần nhớ

1. **Từ khóa** `partial` **:** Class ViewModel bắt buộc phải là `public partial class` để Source Generator có thể nối thêm file code tự sinh vào class của cậu.
2. **Quy ước đặt tên biến:** Đặt biến `private` theo kiểu `_camelCase` hoặc `camelCase` (ví dụ `_firstName` hoặc `firstName`), Toolkit sẽ tự đổi thành `FirstName` ở public property.
3. **Quy ước đặt tên hàm Command:** Hàm tên `Login` hoặc `LoginAsync` khi gắn `[RelayCommand]` đều tự động sinh ra `LoginCommand` để bind vào `<Button Command="{Binding LoginCommand}"/>` .

Nếu cậu muốn đào sâu thêm các tính năng thực chiến của Toolkit này:

Xem cách cập nhật Property phụ thuộc ([NotifyPropertyChangedFor])

↪

Tìm hiểu cách giao tiếp giữa các ViewModel (WeakReferenceMessenger)

↪